# Enhancing Split Federated Learning With Worker Clustering and Feature Compression

Yang Xu, *Member, IEEE*, Yunming Liao, Hongli Xu, *Member, IEEE*, Zhiwei Yao, Junhao Cheng, and Chunming Qiao, *Fellow, IEEE*

***Abstract*—Mobile devices contribute more than half of the world's web traffic, providing massive and diverse data for powering various computer vision (CV) applications, such as fall detection and image segmentation. To extract knowledge from remote devices without pulling their raw data, federated learning (FL) has emerged and usually works in a client-server fashion. To relax the computation burdens of training large-scale models on resource-constrained workers, we propose a novel split federated learning (SFL) framework with worker clustering and feature compression, termed *ClusterSFL*. All participating workers are supposed to train only the bottom or top submodel, and are arranged into multiple clusters. In each cluster, the workers collaboratively perform the SFL training procedure in parallel, while the PS only aggregates the cluster-wise entire models rather than massive features. However, considering the critical challenges encountered in on-device intelligent CV applications, such as communication resource limitation, statistical heterogeneity, and system heterogeneity, it is challenging to arrange suitable workers to specific clusters for efficient model training. Herein, we develop an effective worker clustering strategy and feature compression ratio configuration by optimizing the utility functions related to training efficiency and model accuracy. Specifically, ClusterSFL partitions workers with appropriate feature compression ratios into different clusters under the heterogeneity restrictions, thereby promoting model accuracy and training efficiency. Meanwhile, ClusterSFL adaptively assigns diverse local updating frequencies for each cluster to further address system heterogeneity. Extensive experiments are conducted on a physical platform with 80 NVIDIA Jetson devices, and the experimental results show that ClusterSFL reduces traffic consumption by about 22%∼63%, speeds up model training by about $1.29\times \sim 2.83\times$, and improves model accuracy by about 2%∼27% in heterogeneous scenarios, compared to the baselines.**

***Index Terms*—Edge computing, split federated learning, system heterogeneity, statistical heterogeneity.**

Received 15 November 2024; revised 1 May 2025; accepted 9 June 2025. Date of publication 28 October 2025; date of current version 23 January 2026. This work was supported by the National Science Foundation of China (NSFC) under Grant 62472401, Grant 62132019, and Grant 624B2136. An earlier version of this paper was presented at the 2024 Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (ACM MobiCom) [DOI: 10.1145/3636534.3690665]. *(Corresponding author: Hongli Xu.)*

Yang Xu, Yunming Liao, Hongli Xu, Zhiwei Yao, and Junhao Cheng are with the School of Computer Science and Technology, University of Science and Technology of China, Hefei, Anhui 230027, China, and also with Suzhou Institute for Advanced Research, University of Science and Technology of China, Suzhou, Jiangsu 215123, China (e-mail: xuyangcs@ustc.edu.cn; ymliao98@mail.ustc.edu.cn; xuhongli@ustc.edu.cn; zhiweiyao@mail.ustc.edu.cn; ustc_cjh@mail.ustc.edu.cn).

Chunming Qiao is with the Department of Computer Science and Engineering, University at Buffalo, The State University of New York, Buffalo, NY 14260 USA (e-mail: qiao@buffalo.edu).

Digital Object Identifier 10.1109/JSAC.2025.3623173

## I. Introduction

With the development of Smart Home, various privacy data (*e.g.*, monitoring data, sensor data) are collected by home devices and stored in the smart gateways without sharing with others [2], [3]. However, these privacy data are extremely valuable and urgently needed for training to further improve the performance of on-device intelligent computer vision (CV) applications, such as fall detection [4], fire surveillance [5], and intrusion detection [6]. To extract knowledge in the vast amount of privacy data, *edge AI* has been proposed and become a dominant tool [7], [8]. One of the popular techniques in edge AI is federated learning (FL), which trains a globally-shared model through collaboration among smart gateways in the data-parallel fashion [9], [10], [11], [12].

In FL, each worker (*e.g.*, an edge device) is responsible for training an entire model and periodically pushes/pulls the updated/aggregated model to/from the parameter server (PS) until model convergence. To boost the performance of AI applications/services, it is necessary and practical to augment the parameters of deep learning models, *e.g.*, large language models (LLMs) with transformer architectures [13], [14], [15], [16]. However, due to the hardware limitations of the resource-constrained workers, which usually are only equipped with 1∼30TOPS computing power and 1∼8GB memory, the requirements of high computing power and large memory for model training hinder each worker from training a complete large-scale model [17], [18], [19], [20].

Hence, split federated learning (SFL) has been proposed by incorporating both data parallelism and model parallelism to train large-scale models such as large CNN models and LLMs [18], [21], [22], [23]. SFL splits an entire model into two submodels, *i.e.*, bottom submodel and top submodel, at the split layer. The bottom submodel is trained on the resource-constrained workers, while the top submodel is offloaded to the PS. Then the workers perform the training procedure by continuously exchanging the features (also called activations or smashed data) and gradients with the PS. However, considering the contradiction between the limited bandwidth of PS and numerous workers, the PS may become the system bottleneck, leading to the risk of network congestion and poor scalability [10].

To avoid the communication bottleneck of the PS, we propose a novel SFL framework, termed *ClusterSFL*, which partitions workers into multiple clusters, and encourages each cluster to collaboratively perform the SFL training

0733-8716 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:02:24 UTC from IEEE Xplore. Restrictions apply.

procedure and exchange entire models with the PS. Within each cluster, one worker (denoted as the top worker) maintains the top submodel while the remaining workers (denoted as bottom workers) train the bottom submodels. Together, they perform local updating by exchanging features and gradients. After local updating, the top worker aggregates the bottom submodels and splices with its top submodel to generate a cluster-wise entire model, which is further delivered to the PS for aggregation. With the benefits of split learning, ClusterSFL can not only release the computation burden on resource-constrained workers for training large-scale models, but also avoid the communication bottleneck and reduce the network traffic of the PS.

However, apart from the resource limitation, ClusterSFL still suffers from two other critical challenges in practical applications. 1) ***Statistical Heterogeneity.*** Since the local data collected by workers depend on their user pReferences and are not shared with others for privacy concerns in Web 3.0, resulting in non-independent and identically distributed (non-IID) data across all workers *i.e.*, statistical heterogeneity [28], [29], [30], [31]. The non-IID data slows down the convergence rate and even compromises the accuracy of the trained models [25], [32], [33]. 2) ***System Heterogeneity.*** Workers generally are configured with varying and limited capabilities in edge networks. The computing and communication capabilities of workers could differ from each other by more than tenfold times [34], [35], [36]. System heterogeneity poses significant influences on synchronous training processes, as fast workers may be forced to wait for slow ones, leading to increased waiting time and decreased training efficiency [25], [37].

To this end, ClusterSFL develops a more effective worker clustering strategy by introducing feature compression and feature mixing upon the distinct properties of SFL to tackle the heterogeneity issues simultaneously, which is fundamentally different from the clustering strategy in ParallelSFL and existing FL works. The comparison of ClusterSFL and other related works is summarized in Table I (more details in Sec. II), which shows the advantages of our ClusterSFL. On one hand, to address system heterogeneity, it is necessary to shrink the waiting time across workers in each cluster as far as possible, by organizing workers with similar computing/communication capacities and appropriate feature compression ratios into the same cluster. On the other hand, motivated by the existing works [19], if we mix the features from bottom workers of each cluster (denoted as *feature mixing*) to form a mixed feature sequence, which is approximately equivalent to the features derived from an IID data samples, both the top model and the bottom models will be updated along the reasonably optimal direction to deal with statistical heterogeneity. Herein, we introduce the KL-divergence to measure the gap between the data distribution of each cluster and IID, and optimize the gap for feature mixing in each cluster to enhance model accuracy in ClusterSFL. However, it is intricate to develop an effective worker clustering strategy and adaptive feature compression ratio configuration under the heterogeneity restrictions, so as to balance the trade-off between training efficiency and model accuracy. Meanwhile, we assign diverse and appropriate local updating frequencies for the clusters to further reduce the waiting time across clusters. In a nutshell, our main contributions are as follows:

- We propose a novel SFL framework, named ClusterSFL, which is designed to address the critical challenges encountered in on-device intelligent CV applications, such as communication resource limitation, statistical heterogeneity and system heterogeneity, by effective clustering strategy with feature mixing and feature compression upon the distinct properties of SFL.
- We define the utility functions with the heterogeneity restrictions to serve as the comprehensive metrics for estimating waiting time and data distribution of worker clusters. Upon this, we develop the clustering strategy and feature compression ratio configuration to simultaneously improve training efficiency and enhance model accuracy.
- We evaluate the performance of ClusterSFL through a physical platform with totally 80 NVIDIA Jetson edge devices. The experimental results show that ClusterSFL can reduce the traffic consumption by about 22%$\sim$63%, speed up the model training by about $1.29\times \sim 2.83\times$, and improve the model accuracy by about 2%$\sim$27% in heterogeneous scenarios, compared to the baselines.

The rest of the paper is organized as follows. Sec. II reviews some related works of SFL and FL. Sec. III introduces the background of FL as well as SFL and present our proposed SFL framework. Sec. IV elaborates the detailed design of our framework. In Sec. V, we perform extensive experiments to evaluate our framework. Finally, the whole paper is concluded in Sec. VI.

## II. Related Work

### A. Related Works of Federated Learning

Prior to the emergence of SFL, many solutions to address the heterogeneity challenges [25], [26], [35], [36] have been studied in typical FL scenarios. In order to alleviate the negative effect of system heterogeneity, Diao et al. [38] propose HeteroFL, which enables the training of heterogeneous local models with varying computation complexities on different workers. Liu et al. [39] investigate the effects of batch size on training performance of FL while Ma et al. [40] address the synchronization barrier via adaptive batch sizes and scaled learning rates. Besides, Xu et al. [36] investigate to optimize the local updating frequency of different workers, where the workers with high computing/communication capacities are assigned with larger local updating frequencies. To deal with statistical heterogeneity, Sattler et al. [24] propose IFCA, the clustered FL method to alternate between estimating the cluster identities and minimizing the loss functions, while Shin et al. [41] propose FedBalancer to actively select clients' important training samples. In addition, other works [25], [26], [35] propose to employ worker selection to simultaneously address system and statistical heterogeneity. Specifically, Li et al. [25] develop PyramidFL, a fine-grained worker selection strategy that focuses on the divergence between the selected workers and the remaining workers to fully exploit the computing resource and data of different workers. In addition, Luo et al. [26] propose AdaSampling and design

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:02:24 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF OUR CLUSTERSFL WITH OTHER RELATED WORKS

| | Model Spliting | Statistical Heterogeneity | System Heterogeneity | Bottleneck of PS | Communication Constraint |
|---|---|---|---|---|---|
| FedAvg [12] | × | × | × | × | × |
| IFCA [24] | × | √ | × | √ | × |
| PyramidFL [25] | × | √ | √ | × | × |
| AdaSampling [26] | × | √ | √ | × | × |
| LocFedMix-SL [20] | √ | × | × | × | × |
| AdaSFL [27] | √ | × | √ | × | × |
| ParallelSFL [1] | √ | √ | √ | √ | × |
| **ClusterSFL** | √ | √ | √ | √ | √ |

an adaptive worker sampling algorithm, which tackles both system and statistical heterogeneity to minimize the wall-clock training time. There are also many existing works [24], [42], [43] on clustering strategies in FL to deal with system or statistical heterogeneity. However, those FL researches still face the challenge of training large-scale models on resource-constrained workers. Furthermore, those FL researches can not be directly applied for SFL and ClusterSFL, since workers maintaining only the bottom models must exchange smashed data/gradients with the PS (or the top worker) keeping the top model continuously.

### B. Related Works of Split Federated Learning

The previous vanilla SL methods [44], [45] are proposed to help workers collaboratively train DL models without sharing sensitive raw data in domains such as health care and finance. At that time, the server needs to communicate with workers one by one to complete model training, leading to poor flexibility and scalability. By incorporating FL with SL, SplitFed [21] firstly demonstrates the feasibility and superiority of SFL, and aggregates bottom models after each local updating. Such frequent aggregation results in high network traffic consumption. To save the traffic consumption, LocSplitFed [22] allows the workers not to send features to the PS by using local-loss-based training, which cannot update the top model in time and results in much computing resource. Then, LocFedMix-SL [20] is implemented to maintain all the benefits of SplitFed and LocSplitFed with fixed local updating frequency, but still cannot fully utilize the capacities of heterogeneous workers. In addition, Kim et al. [46] and Joshi et al. [47] propose SFL approaches without aggregating bottom models to reduce the traffic consumption and accelerate model training, but sacrificing model accuracy. There are also related works introducing feature compression [48], [49], [50] to further reduce traffic consumption. Concretely, Zheng et al. [50] propose randomized top-k sparsifcation for feature compression through analysis of the cut layer size reduction and top-k sparsifcation. To enhance the application of SFL, Han et al. [51] propose a joint training and inference strategy, termed SplitGP, with the key idea of optimizing the bottom model to have personalization capability tailored to its main task, while training the top model to have generalization capability for handling out-of-distribution tasks. The existing SFL works mainly focus on training large-scale DL models on resource-constrained workers. Although Liao et al. [27] present an advanced solution (*i.e.*, AdaSFL), which assigns adaptive and diverse batch sizes for different workers to address system heterogeneity, AdaSFL still cannot deal with the statistical heterogeneity. Despite these notable advancements, none of the above SFL works have yet explored to simultaneously address system and statistical heterogeneity. Recently, Liao et al. [19] propose MergeSFL to incorporate feature merging and batch size regulation to overcome the challenges of system and statistical heterogeneity. However, in MergeSFL, all the workers continuously communicate with the PS, leading to the system bottleneck of PS and poor scalability.

## III. PRELIMINARY

### A. Training Procedure of Federated Learning

There are $N$ workers and a parameter server (PS) constituting an edge network, where FL is implemented to perform the learning tasks through a decentralized collaboration among the participating workers. In FL, the PS maintains a globally shared model $\boldsymbol{w}$ with complete structure, while each worker $i$ ($\in [N]$) holds its local data $\mathbb{D}_i$ and trains a local model $\boldsymbol{w}_i$ (*i.e.*, a replica of the global model $\boldsymbol{w}$). The objective of FL is to find the optimal model $\boldsymbol{w}^*$ that minimizes the loss function, which can be defined as:

$$\min_{\boldsymbol{w}} F(\boldsymbol{w}) \triangleq \frac{1}{N}\sum_{i=1}^{N} F_i(\boldsymbol{w}_i) \tag{1}$$

where $F_i(\boldsymbol{w}_i) = \frac{1}{|\mathbb{D}_i|}\sum_{x\in\mathbb{D}_i} \ell(x;\boldsymbol{w}_i)$ is the local loss function of worker $i$, and $\ell(x;\boldsymbol{w}_i)$ is the loss with respect to the model $\boldsymbol{w}_i$ and the data sample $x$. For ease of expression, we list some important notations of this paper in Table II.

Due to the inherent complexity of most DL tasks, it is both essential and functional to employ the gradient descent algorithms [40], [52] to solve Eq. (1). In each aggregation round, worker $i$ downloads the global model $\boldsymbol{w}$ from the PS and independently trains local model $\boldsymbol{w}_i$ using its own data $\mathbb{D}_i$. The PS aggregates the models from all workers and obtains the updated global model $\boldsymbol{w} = \frac{1}{N}\sum_{i=1}^{N} \boldsymbol{w}_i$ and moves on to the next round. In doing so, the global model can acquire knowledge from the local data of different workers without leaking their data privacy. However, training large-scale models in typical FL remains challenging, as it imposes a

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:02:24 UTC from IEEE Xplore. Restrictions apply.

TABLE II
KEY NOTATIONS

| Notation | Semantics |
|---|---|
| $N$ | number of workers |
| $\mathbb{D}_i$ | local dataset of worker $i$ |
| $\boldsymbol{w}_{p,c}$ | the top submodel on top worker in cluster $c$ |
| $\boldsymbol{w}_{b,i}$ | the bottom submodel on bottom worker $i$ |
| $C^h$ | the number of clusters in round $h$ |
| $N_c$ | the number of bottom workers in cluster $c$ |
| $g_i^{h,k}$ | the feature from bottom worker $i$ at iteration $k$ in round $h$ |
| $\gamma_i^{h,k}$ | the feature compression ratio of $g_i^{h,k}$ |
| $G_c^{h,k}$ | the mixed feature sequence of cluster $c$ at iteration $k$ in round $h$ |
| $B_i^h$ | the available ingress bandwidth of worker $i$ in each round $h$ |
| $\mu_{b,i}^{h,k}$ | the computing time of one iteration on $\boldsymbol{w}_{b,i}$ for worker $i$ at iteration $k$ in round $h$ |
| $\beta_{i,j}^{h,k}$ | the transmission time of $g_i^{h,k}$ from worker $i$ to worker $j$ at iteration $k$ in round $h$ |
| $\Phi_c$ | the label distribution of bottom workers in cluster $c$ |
| $\mathcal{W}_c^{h,k}$ | the average waiting time of cluster $c$ at iteration $k$ in round $h$ |
| $\mathcal{W}^h$ | the average waiting time of all clusters in round $h$ |
| $\tau_c^h$ | the local updating frequency of cluster $c$ in round $h$ |

significant computation burden on resource-constrained workers and entails substantial communication overhead for model exchange between the PS and workers [20], [22].

### B. Training Procedure of Split Federated Learning

SFL serves as an alternative solution to process complex real-world data using a large-scale model. The fundamental concept of SFL is to split the full model $\boldsymbol{w}$ into two submodels at the split layer, denoted as $\boldsymbol{w} = [\boldsymbol{w}_b, \boldsymbol{w}_p]$, where $\boldsymbol{w}_b$ represents the bottom submodel and $\boldsymbol{w}_p$ represents the top submodel. In the case of a CNN model, the bottom submodel typically comprises the input layer and convolutional layers whereas the top submodel consists of fully-connected layers and the output layer. In SFL, the PS holds the top submodel $\boldsymbol{w}_p$, while each worker $i$ trains a bottom submodel $\boldsymbol{w}_{b,i}$ using its local data $\mathbb{D}_i$. By training only the submodels, the computation burden of workers can be significantly reduced, compared to FL

The basic training process of SFL involves three main stages, *i.e.*, forward/backward propagation of the worker-specific bottom submodels, forward/backward propagation of the top submodel, and global aggregation of bottom submodels on the PS. Firstly, each worker performs forward propagation with a batch of data samples, and delivers the features of the split layer to the PS. Subsequently, the PS conducts forward/backward propagation to update the top submodel.

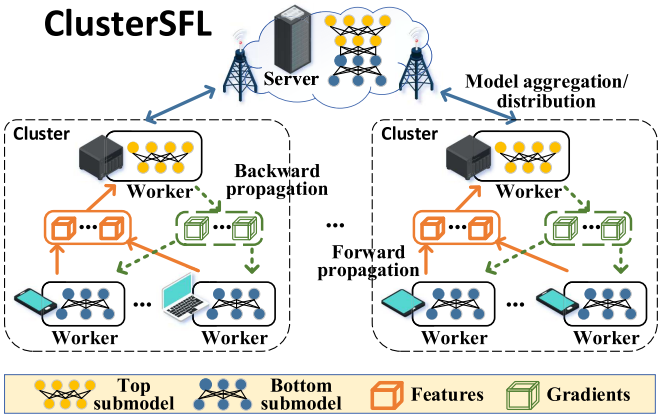



Fig. 1. Illustration of ClusterSFL.

Then, the PS sends the corresponding gradients back to the workers, enabling them to update their bottom submodels through backward propagation. After local updating, the PS aggregates the bottom submodels from all workers and sends the aggregated bottom submodels back to the workers for further training. However, SFL requires continuous exchange of features/gradients between workers and the PS, which consumes the available bandwidth of the PS and generates a substantial amount of traffic workload. Consequently, the PS may become a system bottleneck, leading to the risk of network congestion and poor scalability.

### C. Our Proposed SFL Framework

To alleviate the computing/communication burden on the resource-constrained workers and the PS, we propose a novel SFL framework, called *ClusterSFL*, which is designed to tackle heterogeneity issues. Fig. 1 illustrates the ClusterSFL framework, where we partition the $N$ workers into $C^h$ clusters in aggregation round $h$. In the cluster $c$, there are $N_c$ bottom workers training the bottom submodels and one designated top worker keeping the top submodel. Within each cluster, the bottom workers and the top worker collaborate to perform local updating by exchanging features and gradients. After several local iterations, the top worker aggregates the bottom submodels and splices with its top submodel to generate a cluster-wise entire model, which is further delivered to the PS for aggregation.

In our previous ParallelSFL [1], the top worker directly applies the features of each bottom worker to complete the forward/backward propagation of the top model, and sends the corresponding gradients back to the workers in sequence. However, the features of different bottom workers with non-IID data may hinder the model from being updated along the optimal directions, which reduces model accuracy. To further tackle the non-IID issue, we introduce the strategy of *feature mixing* [19] for each cluster in ClusterSFL. The idea of feature mixing is to mix the features (with small batch sizes) from all bottom workers to form one mixed feature sequence (with a large batch size), which is approximately equivalent to the feature derived from IID mini-batch and is utilized to conduct forward/backward propagation of the top model.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:02:24 UTC from IEEE Xplore. Restrictions apply.

Subsequently, the top worker segments the mixed large-size gradient into multiple small-size gradients corresponding to each bottom worker, and then dispatches the gradients back to the bottom workers to update the bottom models. Considering the advantage of feature mixing, both the top model and bottom models in each cluster will be updated along the relatively right directions in the optimization space, which contributes to improving model accuracy.

Besides, in ParallelSFL, the bottom workers need to frequently exchange features/gradients with the top worker in each cluster, which consumes a lot of communication resource and leads to long communication time. Therefore, it is necessary to perform feature compression for each bottom worker to save communication resource and accelerate the training process. Furthermore, considering the varying and heterogeneous computing/communication capabilities of different bottom workers, we further propose to configure the bottom workers with diverse and appropriate feature compression ratios to reduce the waiting time among bottom workers in each cluster. Concretely, in each cluster, the bottom workers with lower computing/communication capabilities can be configured with larger feature compression ratios to reduce more communication resource, while the bottom workers with higher computing/communication capabilities can be configured with lower feature compression ratios to resever more information.

For each cluster $c$, the bottom worker $i$ performs forward propagation on the bottom submodel $\boldsymbol{w}_{b,i}^{h,k}$ at iteration $k$ in round $h$ as follows:

$$\boldsymbol{w}_{b,i}^{h,k+1} = \boldsymbol{w}_{b,i}^{h,k} - \eta \widetilde{\nabla} F_{b,i}(\boldsymbol{w}_{b,i}^{h,k}) \tag{2}$$

Here, $\widetilde{\nabla} F_{b,i}(\boldsymbol{w}_{b,i}^{h,k}) = \frac{1}{|D_i|} \sum_{x \in D_i} \nabla \ell(x; \boldsymbol{w}_{b,i}^{h,k})$ is the gradient for a certain mini-batch $D_i$ with size $d = |D_i|$, and $\nabla \ell(x; \boldsymbol{w}_{b,i}^{h,k})$ denotes the stochastic gradient given the bottom submodel $\boldsymbol{w}_{b,i}^{h,k}$ and input data sample $x$. Then the bottom worker $i$ delivers its features $g_i^{h,k}$ with feature compression ratio $\gamma_i^{h,k}$ and the corresponding labels to the top worker. In terms of the feature mixing configuration, the top worker will mix the received features from the bottom workers and obtain a mixed feature sequence, which is expected to be the features derived from an IID mini-batch. The mixed feature sequence from bottom workers in the cluster $c$ is denoted as $G_c^{h,k} = [g_i^{h,k}, \ldots, g_j^{h,k}], \forall i, j \in [N_c]$. Thus, the top worker performs forward/backward propagation on the top submodel $\boldsymbol{w}_{p,c}^{h,k}$ with the mixed feature sequence $G_c^{h,k}$ at iteration $k$ in round $h$ as follows:

$$\boldsymbol{w}_{p,c}^{h,k+1} = \boldsymbol{w}_{p,c}^{h,k} - \eta \cdot \widetilde{\nabla} F_{p,c}(\boldsymbol{w}_{p,c}^{h,k}) \tag{3}$$

where $\widetilde{\nabla} F_{p,c}(\boldsymbol{w}_{p,c}^{h,k}) = \frac{1}{N_c \cdot d} \cdot \widetilde{\nabla} \ell(G_c^{h,k}; \boldsymbol{w}_{p,c}^{h,k})$ and $\eta$ is the learning rate of the top submodel. If the bottom workers only deliver their features without labels, the top worker would return the output logits of top model to the workers for calculating loss locally. Then the bottom workers send the loss back to the top worker for calculating the gradients and completing the backward propagation.

After performing backward propagation of the top model, the top worker of cluster $c$ obtains the mixed backpropagated gradients $\widehat{G}_c^{h,k}$ at iteration $k$ in round $h$. In order to correctly update the bottom models of different bottom workers, it is necessary for the workers to obtain the gradients corresponding to their features uploaded at the feature mixing phase. Concretely, the PS first segments the mixed large-size gradients $\widehat{G}^{h,k}$ into multiple small-size gradients $[\hat{g}_i^{h,k}, \ldots, \hat{g}_j^{h,k}], \forall i, j \in [N_c]$ for the corresponding bottom workers in cluster $c$. Then, the PS dispatches the corresponding gradients to the selected workers for completing backward propagation according to Eq. (2).

After local updating, the top worker aggregates the bottom submodels from all bottom workers in cluster $c$ as follows:

$$\boldsymbol{w}_{b,c}^{h} = \frac{1}{N_c} \sum_{i=1}^{N_c} \boldsymbol{w}_{b,i}^{h} \tag{4}$$

Subsequently, the top worker of cluster $c$ aggregates the bottom submodels and splices with its top submodel to generate a cluster-wise entire model $w_c^h = [w_{b,c}^h, w_{p,c}^h]$, which is further delivered to the PS for aggregation. Finally, the PS obtains the updated global model as follows:

$$\boldsymbol{w}^{h+1} = \frac{1}{C^h} \sum_{c=1}^{C^h} w_c^h \tag{5}$$

After that, the PS distributes the updated global model $\boldsymbol{w}^{h+1}$ to the top worker in new clusters and moves on to the next aggregation round.

To tackle the heterogeneity issues and avoid the communication bottleneck, it is crucial to carefully group the workers into proper clusters and configure each worker with appropriate feature compression ratios to enhance model accuracy and improve training efficiency simultaneously. In Sec. IV-C, we develop the worker clustering strategy with feature mixing and adaptive feature compression ratio configuration under the heterogeneity restrictions. However, considering the heterogeneous workers in diverse clusters, the complete time of one local iteration among clusters still varies significantly. In traditional synchronous schemes, if we assign identical local updating frequencies for all clusters, the fast clusters are forced to wait for the slow ones, incurring idle waiting time and significantly destroying the training efficiency. Generally, the clusters with higher performance can be allocated with larger local updating frequencies, while those with lower performance are allocated with smaller ones. Therefore, the waiting time among clusters would be greatly reduced.

## IV. System Design

This section will elaborate the detailed design of ClusterSFL to tackle heterogeneity issues. We firstly present the overview design of ClusterSFL in Sec. IV-A. Then we give the details of worker state collection in Sec. IV-B. The process of making the clustering strategy is presented in Sec. IV-C. Subsequently, we give the local updating frequency configuration in Sec. IV-D. Finally, we present the module of full model updating in Sec. IV-E and discussion of privacy protection in Sec. IV-F.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:02:24 UTC from IEEE Xplore. Restrictions apply.

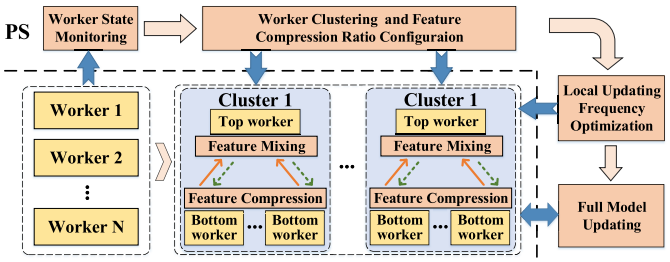



Fig. 2. Overview of ClusterSFL.

### A. Overview

The overall design of ClusterSFL is illustrated in Fig. 2. ClusterSFL consists of four key modules at the PS side, *i.e.*, *worker state monitoring*, *worker clustering and feature compression ratio configuration*, *local updating frequency optimization*, and *full model updating*. At the worker side, there are three key modules, *i.e.*, local updating, feature compression, and feature mixing. At the beginning of each aggregation round, the worker state monitoring module starts to collect the state information (*e.g.*, ingress bandwidth, label distribution, computing and communication capabilities) from all workers. After that, the worker clustering and feature compression ratio configuration module in the PS organizes the workers into suitable clusters and configures the bottom workers of each cluster with adaptive feature compression ratios to tackle heterogeneity issues. Subsequently, the local updating frequency optimization module determines diverse and appropriate local updating frequencies for clusters to further reduce the waiting time across clusters. Meanwhile, the full model updating module distributes the full model to the top worker of each cluster, and then the top worker delivers the bottom submodels to the bottom workers in the cluster. Within each cluster, the bottom workers continuously send the compressed features to the top worker for feature mixing to update the top model at each iteration. After a certain number of local iterations, the top worker aggregates the bottom submodels and splices with its top submodel to generate a cluster-wise entire model in each cluster. Then, the PS aggregates the cluster-wise entire model from the top workers in each cluster, and moves on to the next aggregation round.

### B. Worker State Monitoring

In order to make an effective clustering strategy, it is necessary to monitor the current states of all workers (*e.g.*, ingress bandwidth, label distribution, computing and communication capabilities).

In edge networks, each worker $i$ has a limited available ingress bandwidth $B_i^h$ in each round $h$. The top worker in each cluster typically consumes a significant portion of the bandwidth to exchange features/gradients with bottom workers. To prevent the top worker from becoming a bottleneck, it is instinct to ensure that the occupied bandwidth of the top worker does not exceed the available ingress bandwidth. At the beginning of a certain round, the PS collects and analyzes the statistical distribution of the ingress bandwidth of all workers. Then the PS employs the statistical results as the available ingress bandwidth for each worker.

Secondly, the local data of each cluster together (except the top worker) should be close to IID. Herein, the label distribution, a vector $\mathbf{V} = \{v_j \geq 0, j \in [1, M]\}$ ($\sum_{j=1}^{M} v_j = 1$) to parameterize a categorical distribution of class labels over $M$ classes, is required to assist in worker clustering. In typical SFL, the workers deliver the features with corresponding labels to continue forward propagation of the top submodel on the PS, which enables the PS to directly collect the labels of workers' feature and generates the label distribution $\mathbf{V}_i$ of the mini-batch of worker $i$ [18], [21], [22].

Thirdly, the collection of time-varying computing and communication capacities of workers is necessary for worker clustering and local updating frequency optimization. On one hand, we denote the computing time of one iteration on the bottom submodel and top submodel for worker $i$ at iteration $k$ in round $h$ as $\mu_{b,i}^{h,k}$ and $\mu_{p,i}^{h,k}$, respectively. The relationship between these two parameters is clear and fixed, so that we only monitor $\mu_{b,i}^{h,k}$. On the other hand, $\beta_{i,j}^{h,k}$ is indicated as the transmission time of the feature/gradients between worker $i$ and worker $j$ at iteration $k$ in round $h$. As proxy metrics, we adopt $\mu_{b,i}^{h,k}$ and $\beta_{i,j}^{h,k}$, which can be recorded by the workers directly during model training, to indicate the computing and communication capacities of worker $i$ at iteration $k$ in round $h$, respectively. Prior to starting model training at iteration $k$ in round $h$, the PS collects the latest $\hat{\mu}_{b,i}^{h,k}$ and $\hat{\beta}_{i,j}^{h,k}$ from all workers. Besides, to improve the robustness of the estimation, we introduce the moving average with the historical states of workers [53]. Accordingly, the PS estimates $\mu_{b,i}^{h,k}$ and $\beta_{i,j}^{h,k}$ on worker $i$ at iteration $k$ in aggregation round $h$ by calculating the moving average with $\alpha \in [0, 1]$ (*e.g.*, 0.8 in our experiments) as:

$$\mu_{b,i}^{h,k} = \alpha \cdot \mu_{b,i}^{h,k-1} + (1-\alpha) \cdot \hat{\mu}_{b,i}^{h,k} \tag{6}$$

$$\beta_{i,j}^{h,k} = \alpha \cdot \beta_{i,j}^{h,k-1} + (1-\alpha) \cdot \hat{\beta}_{i,j}^{h,k} \tag{7}$$

Since the size of the information about worker states (*e.g.*, 100-300KB [54]) is much smaller than that of model parameters, it is reasonable to ignore the cost (*e.g.*, bandwidth consumption and time cost) for state monitoring [55].

### C. Worker Clustering and Feature Compression Ratio Configuration

Based on the collected state information, ClusterSFL partitions the workers with adaptive and appropriate feature compression ratios into $C^h$ clusters and determines the top worker for each cluster. The bottom workers in the same cluster perform feature mixing. Besides, each cluster is assigned with diverse and suitable local updating frequency.

In each cluster $c$, there are $N_c$ bottom workers and one designated top worker. Due to the constraint of the available ingress bandwidth $B_c^h$ for the top worker, the number of participating workers $N_c$ to simultaneously train the bottom submodels is limited. We denote the bandwidth occupied by worker $i$ at each iteration as $b_i$. To simplify the problem-solving process, we adopt the constant $b$ to denote the average

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:02:24 UTC from IEEE Xplore. Restrictions apply.

bandwidth occupied by all workers, *i.e.*, $b = \frac{1}{N}\sum_{i=1}^{N} b_i$, and the occupied bandwidth in each cluster is limited as follows:

$$N_c \cdot b \leq B_c^h \tag{8}$$

The size of cluster $c$ depends on the available ingress bandwidth $B_c^h$ of the top worker. Since the local data of the top worker does not contribute to model training, we should control the number of the top worker (*i.e.*, the number of the clusters) as small as possible.

For the cluster $c$, we can formulate the completion time $t_i^{h,k}$ of on the worker $i$ with feature compression ratio $\gamma_i^{h,k}$ at iteration $k$ in round $h$ as:

$$t_i^{h,k} = \mu_{b,i}^{h,k} + \beta_{i,c}^{h,k} \cdot (1 - \gamma_i^{h,k}) + \mu_{p,c}^{h,k} \tag{9}$$

where $\mu_{p,c}^{h,k}$ and $\beta_{i,c}^{h,k}$ separately denote the computing time of one iteration of the top worker and the transmission time of the feature at iteration $k$ from bottom worker $i$ to the top worker in cluster $c$. It is worth noting that the features in ClusterSFL can only be compressed during transmission and need to be restored before updating the top model. All the bottom workers in cluster $c$ send the features with the same size to the top worker at each iteration, so that the top worker consumes the same computing time to perform forward/backward propagation with each feature from the bottom workers. Since the top worker has to simultaneously exchange features/gradients with multiple workers, in order to ensure training efficiency, the number of participating workers $N_c$ in cluster $c$ is limited as:

$$N_c \cdot \mu_{p,c}^{h,k} \leq \max\{\mu_{b,i}^{h,k} + \beta_{i,c}^{h,k} \cdot (1 - \gamma_i^{h,k})\},\ \forall i \in [N_c] \tag{10}$$

Besides, the waiting time of the bottom worker $i$ can be defined as $t_{c,o}^{h,k} - t_i^{h,k}$, where $t_{c,o}^{h,k} = \max\{t_i^{h,k}\}$ $(\forall i \in [N_c])$ denotes the completion time at iteration $k$ in round $h$ for the slowest worker in cluster $c$. Accordingly, the average waiting time of cluster $c$ at iteration $k$ in round $h$ is formulated as:

$$\mathcal{W}_c^{h,k} = \frac{1}{N_c}\sum_{i=1}^{N_c}(t_{c,o}^{h,k} - t_i^{h,k}) \tag{11}$$

To minimize the waiting time of all bottom workers in cluster $c$, we should group the $N_c$ bottom workers with appropriate feature compression ratios into the cluster $c$ so as to align their completion time. Thus, it ensures that the average waiting time will be small enough to mitigate the negative impacts of the synchronization barrier and improve the training efficiency. The regulation rule is expressed as follows:

$$\lfloor \frac{\mu_{b,i}^{h,k} + \beta_{i,c}^{h,k} \cdot (1 - \gamma_i^{h,k})}{\mu_{b,l}^{h,k} + \beta_{l,c}^{h,k} \cdot (1 - \gamma_l^{h,k})} \rfloor = 1 \quad \forall i \in [N_c] \tag{12}$$

where $l$ denotes the index of the fastest bottom worker of cluster $c$ assigned with the default feature compression ratio $\gamma_l^{h,k} = 0$ (*i.e.*, no compression) at iteration $k$ in round $h$. For convenience, we employ top-k sparsification [48] as our compression operator while other compression operators of features (*e.g.*, random-k sparsification [48], autoencoder compressor [49], randomized top-k sparsification [50]) can also be applied in ClusterSFL.

To tackle the non-IID issue, the mixed feature sequence of bottom workers in each cluster needs to be approximately equivalent to the features derived from IID data. We first define the IID distribution as $\Phi_0$. If the data of all workers follows IID distribution, we get $\Phi_0 = \frac{1}{N}\sum_{i=1}^{N} \mathbf{V}_i$, where $\mathbf{V}_i$ is the label distribution of worker $i$. In cluster $c$, the label distribution of $N_c$ workers is:

$$\Phi_c = \frac{1}{N_c}\sum_{i=1}^{N_c} \mathbf{V}_i \tag{13}$$

The mixed feature sequence of the bottom workers in cluster $c$ for feature mixing is expected to meet the requirement that its label distribution $\Phi_c$ is approximately consistent with the IID distribution $\Phi_0$. Then, we introduce the KL-divergence $KL(\Phi_c||\Phi_0)$ [56], [57] to measure the gap between $\Phi_c$ and $\Phi_0$ as follows:

$$KL(\Phi_c||\Phi_0) = \sum_{j=1}^{M} \Phi_c(v_j)\log\frac{\Phi_c(v_j)}{\Phi_0(v_j)} \tag{14}$$

To deal with statistical heterogeneity, it is necessary to control the KL-divergence $KL(\Phi_c||\Phi_0)$ as small as possible.

Taking system and statistical heterogeneity into account, it is challenging to partition these workers into appropriate clusters for feature mixing and configure each worker with appropriate feature compression ratios to enhance model accuracy and improve training efficiency simultaneously. Herein, we propose a greedy algorithm to make the effective clustering strategy and dynamically apply suitable feature compression ratios for the bottom workers at each iteration in the same cluster. Firstly, by the $K$-means algorithm (*e.g.*, $\mathcal{K} = N/5$), according to the KL-divergence of label distribution among workers, we divide the workers with small KL-divergence into the same set and obtain $K$ sets $S_1, \ldots, S_{\mathcal{K}}$ (Line 1-2 of Alg. 1). Next, we greedily select the workers with maximum ingress bandwidth $B_c^h$ from the set $S_\kappa$ with the most workers as the top worker for cluster $c$. We construct the set $A$ including the worker $j$ with the maximum $t_j^{h-1}$ (*i.e.*, the completion time of worker $j$ in round $h-1$) from each set $S_\kappa$ and group workers into the cluster $c$ from set $A$ to make $KL(\Phi_0||\Phi_c)$ smallest under the constraints of Eqs. (8) and (10). Subsequently, we repeat the above operations to further partition the remaining workers and create new clusters, until all workers are grouped into suitable clusters (Line 4-10 of Alg. 1). Upon completion of this process, we obtain the $C^h$ effective clusters in round $h$. Then, for each cluster $c$, we need to assign the bottom workers with appropriate feature compression ratios at each iteration. We firstly select the fastest bottom worker $l$ (*i.e.*, with the minimum $t_l^{h,k}$) and set $\gamma_l^{h,k} = 0$ (*i.e.*, no compression) at iteration $k$ in round $h$ (Line 12-13 of Alg. 1). Finally, we configure feature compression ratios for all bottom workers in cluster $c$ according to the formulation of Eq. (12) (Line 14 of Alg. 1). The time complexity of the greedy algorithm mainly depends on the complexity of the $K$-means algorithm, *i.e.*, $O(N^2)$, in which $N$ denotes the number of total workers. Compared to the time complexity of typical SGD algorithm used in model training, the time complexity of the algorithm can be negligible.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:02:24 UTC from IEEE Xplore. Restrictions apply.

**Algorithm 1** Worker clustering in round $h$

**Input:** $B_i^h$, $\mathbf{V}_i$, $\mu_{b,i}^{h,k}$, $\mu_{p,i}^{h,k}$, $\beta_{i,j}^{h,k}$, $\mathcal{K}$.
**Output:** The worker clustering strategy.
Calculate $KL(\Phi_i||\Phi_j)$ for all workers $i$ and $j$, $i \neq j$.
Divide the workers into $\mathcal{K}$ sets $S_1, \ldots, S_{\mathcal{K}}$ ($S = S_1 \cup \cdots \cup S_{\mathcal{K}}$) by calling the $K$-means algorithm.
Initialize $c = 1$.
**while** $S$ *is not empty* **do**
  Select the worker $i$ with maximum $B_i^h$ from $S_\kappa$ with the most workers as the top worker for cluster $c$.
  Remove worker $i$ from its set.
  Denote the distribution of cluster $c$ as $\Phi_c$.
  Select the worker $j$ with the maximum $t_j^{h-1}$ from each $S_\kappa$ into set $A$.
  Select workers into the cluster $c$ from set $A$ to make $KL(\Phi_0||\Phi_c)$ smallest under the constraints of Eqs. (8) and (10).
  $c \leftarrow c + 1$.
**for** *each cluster* $c$ *at iteration* $k$ **do**
  Select the worker $l$ with the minimum $t_l^{h,k}$ from cluster $c$.
  Set $\gamma_l^{h,k} = 0$
  Configure feature compression ratios for all bottom workers in cluster $c$ according to the formulation of Eq. (12).

### D. Local Updating Frequency Optimization

Additionally, due to the system heterogeneity, the complete time of one iteration among clusters is highly different. We denote the communication time for the top worker in cluster $c$ transmitting the submodels to the PS in round $h$ as $\beta_c^h$. The local updating frequency of cluster $c$ is defined as $\tau_c^h$. Accordingly, the completion time $t_c^h$ of aggregation round $h$ for the cluster $c$ is expressed as:

$$t_c^h = \sum_{k=1}^{\tau_c^h} t_{c,o}^{h,k} + \beta_c^h \tag{15}$$

Besides, the waiting time of cluster $c$ can be defined as $t^h - t_c^h$, where $t^h = \max\{t_c^h\}$ ($\forall c \in [C^h]$) denotes the completion time of aggregation round $h$ for the slowest cluster. Accordingly, the average waiting time of all clusters in round $h$ is formulated as:

$$\mathcal{W}^h = \frac{1}{C^h} \sum_{c=1}^{C^h} (t^h - t_c^h) \tag{16}$$

To minimize the average waiting time, ClusterSFL regulates the local updating frequencies of all clusters so as to align their completion time of one round. Thus, it ensures that the average waiting time will be small enough to mitigate the negative impacts of the synchronization barrier and improve training efficiency. The regulation rule is expressed as:

$$\lfloor \frac{\sum_{k=1}^{\tau_c^h} t_{c,o}^{h,k} + \beta_c^h}{\sum_{k=1}^{\tau_l^h} t_{l,o}^{h,k} + \beta_l^h} \rfloor = 1 \tag{17}$$

where $l$ denotes the index of the fastest clusters assigned with the default maximum local updating frequency $\tau$ in round $h$. According to Eq. (17), we can obtain the specific local updating frequencies for all clusters in round $h$

### E. Full Model Updating

After cluster $c$ performs local updating in round $h$, the top worker firstly aggregates the bottom submodels from other workers according to Eq. (4). Then the top worker splices the bottom submodel and top submodel, and sends the entire model to the PS for global aggregation. Considering the clusters with diverse numbers of workers are configured with different local updating frequencies for model training, the full model in each cluster is updated and trained with varying degrees, which needs adaptive aggregation weight to guarantee the performance of aggregated global model [36], [58], [59]. Therefore, the PS aggregates the entire models from different clusters with adaptive weights related to the local updating frequencies and the size of clusters as follows:

$$\boldsymbol{w}^h = \sum_{c=1}^{C^h} \frac{N_c \cdot \tau_c^h \cdot \boldsymbol{w}_c^h}{\sum_{c=1}^{C^h} N_c \cdot \tau_c^h} \tag{18}$$

The aggregated global model is stored in the PS and will be distributed to future clusters to continue further training, or be used for AI tasks.

### F. Discussion of Privacy Protection

In existing SFL literature [21], [44], [45], there are two ways, *i.e.*, sharing labels and returning logits (without sharing labels), to complete the model updating, which can also be applied in ClusterSFL. Generally, the bottom workers usually deliver the features with corresponding labels to continue forward propagation of the top model on the top worker. However, if the bottom workers are unwilling or not permitted to share labels, the top worker can return the output logits of the top model to the workers for calculating the loss locally. Then the bottom workers send the loss values back to the top worker to derive the gradients and complete the backward propagation. Moreover, the existing privacy-preserving techniques such as Differential Privacy [60], [61], [62] and Homomorphic Encryption [63] can be employed to further protect the privacy of the feature in ClusterSFL. In this paper, we focus on efficient model training on resource-constrained workers in the case of system and statistical heterogeneities, which is orthogonal to privacy protection.

## V. Evaluation

This section will describe the experimental settings and experimental results in detail. We firstly present the experimental settings in Sec. V-A. Then we present the overall performance of ClusterSFL and the baselines in Sec. V-B.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:02:24 UTC from IEEE Xplore. Restrictions apply.

TABLE III
DEVICE TECHNICAL SPECIFICATIONS

| | AI Performance | GPU Type |
|---|---|---|
| Jetson TX2 | 6 TOPs | 256-core Pascal |
| Jetson NX | 21 TOPs | 384-core Volta |
| Jetson AGX | 32 TOPs | 512-core Volta |
| | **CPU Type** | **ROM** |
| Jetson TX2 | Denver 2 and ARM 4 | 8 GB LPDDR4 |
| Jetson NX | 6-core Carmel ARM 8 | 8 GB LPDDR4x |
| Jetson AGX | 8-core Carmel ARM 8 | 32 GB LPDDR4x |
| | **CPU Frequency** | **GPU Frequency** |
| Jetson TX2 | 2.0GHz | 1.12GHz |
| Jetson NX | 1.9GHz | 1.1GHz |
| Jetson AGX | 2.2GHz | 1.37GHz |

Subsequently, the effect of non-IID levels of ClusterSFL and the baselines is presented in Sec. V-C. Finally, we present the effect of key strategies of ClusterSFL in Sec. V-D and the effect of system scales of ClusterSFL and the baselines in Sec. V-E.

### A. Experimental Settings

*1) System Deployment:* We conduct extensive experiments to evaluate the performance of ClusterSFL on an edge computing hardware prototype system. Specifically, we employ a deep learning GPU workstation as the PS, which is equipped with an Intel(R) Core(TM) i9-10900X CPU, four NVIDIA GeForce RTX 2080Ti GPUs and 256 GB RAM. In addition, we specify 80 NVIDIA Jetson kits,[1] including 30 Jetson TX2 devices, 40 Jetson NX devices, and 10 Jetson AGX devices, as workers to construct a heterogeneous system. The detailed technical specifications of Jetson TX2, NX, and AGX are listed in Table III. Notably, the TX2 showcases a 256-core Pascal GPU with 6 TOPs and a CPU cluster consisting of a 2-core Denver2 and a 4-core ARM CortexA57. The NX is outfitted with a 384-core NVIDIA Volta GPU with 21 TOPs and a 6-core NVIDIA Carmel ARMv8.2 CPU. Jetson Xavier NX dramatically enhances the NVIDIA software stack over 10× the performance of Jetson TX2. Lastly, the AGX stands out with a 512-core NVIDIA Volta GPU with 32 TOPs and an 8-core NVIDIA Carmel ARMv8.2 CPU. Besides, the computing power of these devices is at the same level as the current mainstream smart gateways.

In the experiments, we build the software platform based on Docker Swarm [64], [65] and the PyTorch deep learning library [66]. The Docker Swarm, a distributed software development kit, facilitates the construction of a distributed system and enables the monitoring of each device's operational status. The PyTorch library facilitates the implementation of model training on devices. Additionally, to streamline communication among devices, we implement MPI (Message Passing Interface) [67], which includes a collection of sending and receiving functions.

*2) Settings of System Heterogeneity:* To enable the workers with heterogeneous computing and communication capabilities, we present the following experimental settings.

*A. For computation:* All the Jetson TX2, NX, and AGX devices can be configured to work with different modes, which specifies the number of working CPUs and the frequency of CPU/GPU for the devices to work with different computing capacities. Specifically, TX2 can work in four modes each while NX and AGX work in one of eight modes. Devices working in different modes exhibit diverse capabilities. For instance, the AGX with the highest performance mode (*i.e.*, mode 0 of AGX) achieves training by 100× faster than the TX2 with the lowest performance mode (*i.e.*, mode 1 of TX2). To further reflect the time-varying on-device resource, we randomly change the modes for devices every 20 rounds.

*B. For communication:* All devices are connected to the PS via WiFi routers. We arrange 80 devices into four groups, each containing 20 devices. These groups are then placed at different locations, *i.e.*, 2m, 8m, 14m, and 20m away from the WiFi routers. Due to random channel noise and competition among devices, the bandwidth between the PS and devices dynamically varies during the training. The bandwidth of devices is measured by iperf3 [68], which dynamically fluctuates between 1Mb/s and 30Mb/s during training.

*3) Applications and Models:* We firstly evaluate the performance of ClusterSFL on two typical CV applications (*e.g.*, object recognition and image classification) with two typical datasets and DNN models. To further verify the versatility of ClusterSFL, we also add two representative datasets and models from two additional applications, *i.e.*, speech recognition and natural language processing in the experiments.

*A. Object recognition:* We adopt the CIFAR-10 dataset [69] for the evaluation, which is an image dataset composed of 60,000 32×32 color images (50,000 for training and 10000 for test) across 10 categories. We utilize an 8-layer AlexNet with a size of 136MB [70] for CIFAR-10. The AlexNet is composed of three $3 \times 3$ convolutional layers, one $7 \times 7$ convolutional layer, one $11 \times 11$ convolutional layer, two fully-connected hidden layers, and one softmax output layer.

*B. Image classification:* ImageNet [71] is a dataset for image recognition that consists of 1,281,167 training images, 50,000 validation images, and 100,000 test images from 1000 categories. To adapt to the resource-constrained workers, we create a subset of ImageNet, called IMAGE-100, which contains 100 out of 1,000 categories. We adopt a famous large model VGG16 with a size of 321MB [72] for the complex task, which consists of 13 convolutional layers with the kernel of $3 \times 3$, two fully-connected layers, and a softmax output layer.

*C. Speech recognition:* The Google Speech dataset [73] (expressed as Speech for short) is adopted for the task of speech recognition, which allows a computer or device to recognize and interpret spoken language. The dataset includes 85,511 and 4,890 audio clips for training and test, respectively. The model trained on Speech is a plain CNN model with four 1-D convolutional layers and one fully-connected layer.[2]

[1] https://docs.nvidia.com/jetson/

[2] https://pytorch.org/tutorials/intermediate/speech_command_classification_with_torchaudio_tutorial.html

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:02:24 UTC from IEEE Xplore. Restrictions apply.

*D. Natural language processing:* We utilize the Question-answering Natural Language Inference (QNLI) dataset [74], a classification dataset consisting of question-sentence pairs, in this application. There are 104,743 samples for training and 5,463 samples for test in the QNLI dataset. We train a RoBERTa model with a size of 501MB [13] on QNLI dataset. The RoBERTa is composed of 12 transformer layers, a $768 \times 2$ fully-connected layer, and a softmax output layer.

*4) Settings of Statistical Heterogeneity:* In the experiments, training samples of each worker are drawn independently by a vector $\mathbf{v}$. To create non-IID datasets, we draw from a Dirichlet distribution [75], [76], *i.e.*, $\mathbf{v} \sim Dir(\delta \mathbf{q})$, where $\mathbf{q}$ characterizes a prior class distribution, and $\delta > 0$ is a concentration parameter controlling the identicalness among workers. With $\delta \to \infty$, all workers have identical distributions to the prior class distribution (*i.e.*, IID); with $\delta \to 0$, each worker holds data samples from one class, which indicates a high degree of statistical heterogeneity. We specify 6 values (*e.g.*, $\infty$, 1, 0.5, 0.25, 0.2, 0.1) for $\delta$ to generate different data distributions that cover a spectrum of identicalness, and define $p = 1/\delta$ (*i.e.*, $p = 0, 1, 2, 4, 5, 10$) to quantify the non-IID levels. The degree of statistical heterogeneity increases as $p$ increases, and $p = 0$ is a special case of IIDness.

*5) Baselines:* We measure the effectiveness of ClusterSFL through a comparison with four approaches.

*A. LocFedMix-SL:* [20] is a typical SFL approach, which proposes to reduce the aggregation frequency of bottom sub-models to save the traffic consumption, but can not fully utilize the capacities of heterogeneous workers.

*B. AdaSampling:* [26] is an advanced FL approach that focuses on designing an adaptive worker sampling algorithm to tackle both system as well as statistical heterogeneity and minimize the wall-clock convergence time.

*C. AdaSFL:* [27] is a state-of-the-art SFL approach, which assigns adaptive local updating frequency and diverse batch sizes for heterogeneous workers to enhance the training efficiency without addressing statistical heterogeneity.

*D. ParallelSFL:* [1] is a novel SFL framework, which is designed to overcome the resource limitation and track heterogeneity issues by effective cluster partitioning with a utility function under the heterogeneity restrictions. ClusterSFL is an extension of ParallelSFL.

*6) Metrics:* We adopt the following metrics to evaluate the performance of ClusterSFL and the baselines.

*A. Test accuracy:* reflects the accuracy of the models trained by different approaches on the test datasets, and is measured by the proportion of the data correctly predicted by the models to all the test data. We evaluate the test accuracy of the global model in each round, and record the final accuracy for all approaches.

*B. Training time:* is denoted as the total wall clock time taken for training a model to achieve a target accuracy. For a fair comparison, we set the target accuracy as the achievable accuracy by all approaches. In addition, we also record the average waiting time of all workers to reflect the training efficiency of different approaches.

*C. Network traffic:* is calculated by summing the traffic for transmitting models or features between the PS and workers or between the bottom workers and top workers when achieving a target accuracy.

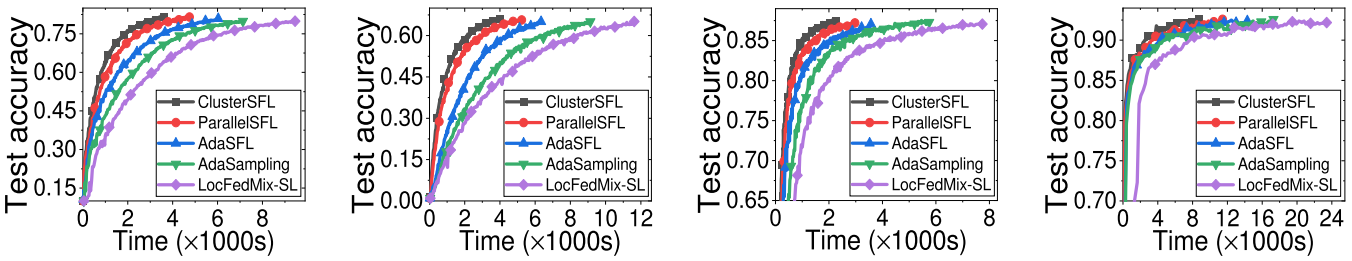
*7) Experimental Parameters:* By default, each set of experiments will run 100 aggregation rounds for RoBERTa on QNLI, and 250 aggregation rounds for CNN on Speech, AlexNet on CIFAR-10, and VGG16 on IMAGE-100. The learning rates and decay rates for CNN, AlexNet, and VGG16 are identical, and are initialized as 0.1 and 0.993 [77], [78], respectively, while for RoBERTa, the learning rate is initialized as 0.001 without decay rate. Besides, the batch size is set as 16 for RoBERTa and 64 for the remaining three models. For the SFL approaches, we separately split the CNN, AlexNet, VGG16, and RoBERTa at the 4th, 5th, 13th, and 3rd layer [27].

## B. Overall Performance

Firstly, we conduct a set of experiments on the four IID datasets to evaluate the performance of ClusterSFL and the baselines. The training processes of five approaches are presented in Fig. 3. By the results, all the approaches achieve similar test accuracy eventually on the four datasets. However, ClusterSFL achieves the fastest convergence, followed by AdaSFL, which is much faster than the other approaches on all the four datasets. For instance, by Fig. 3(a), for AlexNet on CIFAR-10, ClusterSFL can separately speed up training by about 1.29×, 1.76×, 2.14× and 2.83×, compared to ParallelSFL, AdaSFL, AdaSampling and LocFedMix-SL, respectively. AdaSFL with adaptive and diverse batch sizes for heterogeneous workers speeds up the convergence rate, while ParallelSFL with adaptive worker clustering and local updating frequency optimization overcomes the system heterogeneity and achieves the faster convergence. As the extension of ParallelSFL, ClusterSFL assigns adaptive and suitable feature compression ratios to the bottom workers to further reduce the waiting time in the clusters and achieves the fastest convergence. Specifically, for VGG16 on IMAGE-100, as shown in Fig. 3(b), ClusterSFL reduces the total training time by about 23%, 37%, 56%, and 65%, compared to the baselines (*i.e.*, ParallelSFL, AdaSFL, AdaSampling and LocFedMix-SL). Similarly, as illustrated in Fig. 3(c), ClusterSFL takes 2,113s to achieve 87% accuracy for CNN on Speech, while ParallelSFL, AdaSFL, AdaSampling and LocFedMix-SL consume 2,893s, 3,267s, 4,877s and 6,878s, respectively. Moreover, by Fig. 3(d), ClusterSFL takes 8,456s to achieve 92% accuracy for RoBERTa on QNLI, while ParallelSFL, AdaSFL, AdaSampling and LocFedMix-SL consume 10,838s, 13,635s, 16,812s and 22,565s, respectively. These results demonstrate the advantage of ClusterSFL with effective worker clustering and adaptive feature compression in addressing system heterogeneity.

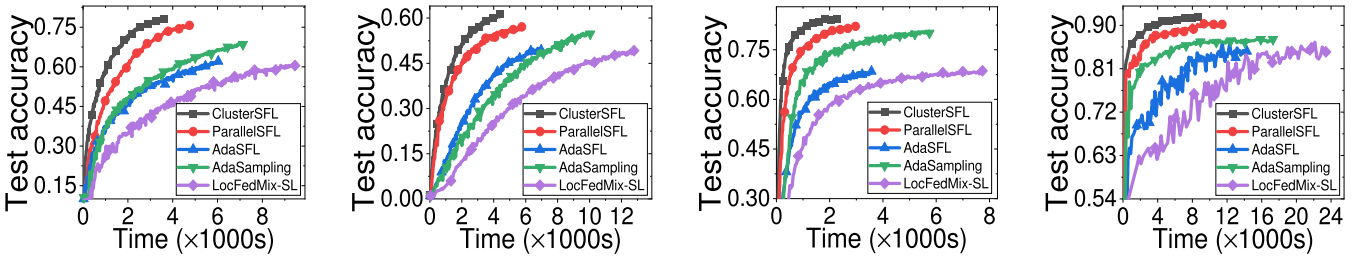
Secondly, we also conduct a set of experiments of these approaches on the four datasets with non-IID level $p = 10$, and the results are presented in Fig. 4. We observe that all the approaches maintain a similar convergence rate as that in the IID scenario, but suffer from varying degrees of accuracy degradation. However, ClusterSFL with adaptive worker clustering and feature mixing achieves the highest

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:02:24 UTC from IEEE Xplore. Restrictions apply.



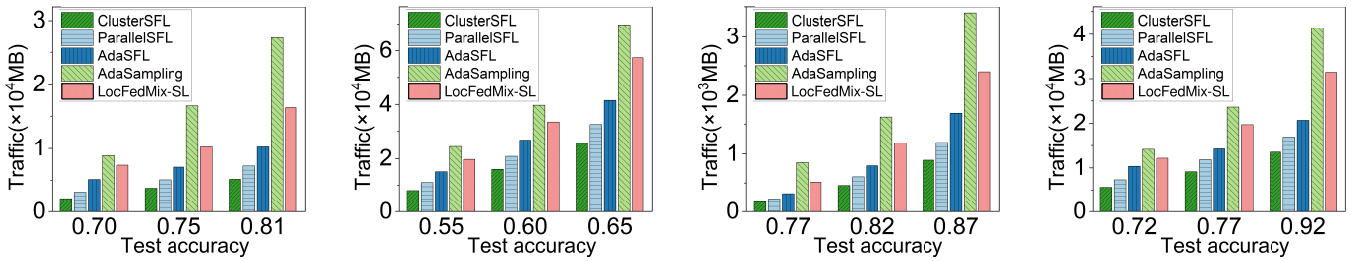

(a) CIFAR-10 (b) IMAGE-100 (c) Speech (d) QNLI

Fig. 3. Test accuracy of five approaches on the four IID datasets.




(a) CIFAR-10 (b) IMAGE-100 (c) Speech (d) QNLI

Fig. 4. Test accuracy of five approaches on the four non-IID datasets.



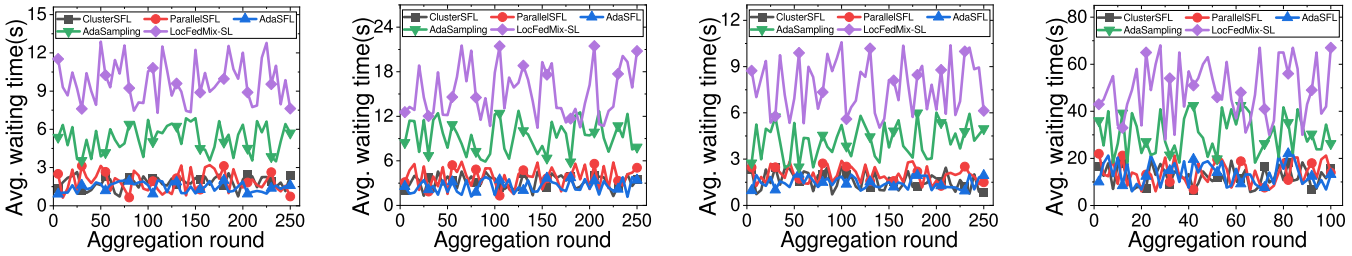

(a) CIFAR-10 (b) IMAGE-100 (c) Speech (d) QNLI

Fig. 5. Network traffic consumption of five approaches when achieving different target accuracies.

accuracy among these approaches. For instance, as shown in Fig. 4(a), for AlexNet on CIFAR-10 with the same training time of 3,600s, ClusterSFL improves the test accuracy by about 6.34%, 40.38%, 33.57% and 74.31%, compared to ParallelSFL, AdaSFL, AdaSampling and LocFedMix-SL, respectively. AdaSampling with adaptive worker sampling speeds up the convergence rate and improves test accuracy while ParallelSFL with adaptive worker clustering and model splitting achieves the higher accuracy than AdaSampling. In addition, by introducing the strategy of feature mixing, ClusterSFL can tackle both system and statistical heterogeneity better than ParallelSFL. Specifically, Fig. 4(b) illustrates that ClusterSFL separately improves the final accuracy by about 5.56%, 23.43%, 12.74% and 23.73% for VGG16 on IMAGE-100, compared to ParallelSFL, AdaSFL, AdaSampling and LocFedMix-SL, respectively. Similarly, by Fig. 4(c), ClusterSFL achieves 84.39% accuracy in 2,359s for CNN on Speech, while ParallelSFL, AdaSFL, AdaSampling and LocFedMix-SL takes 2,972s, 3,567s, 5,747s and 7,737s to reach the accuracy of 82.1%, 68.64%, 80.07% and 68.23%, respectively. Moreover, as shown in Fig. 4(d), ClusterSFL achieves 91.68% accuracy in 8,798s for RoBERTa on QNLI, while ParallelSFL, AdaSFL, AdaSampling and LocFedMix-SL take 11,445s, 14,293s, 17,306s, 23,403s, and 30,724s to reach the accuracy of 90.11%, 84.51%, 87.33% and 84.62%, respectively. These results demonstrate that ClusterSFL with feature mixing is effective in simultaneously tackling the system and statistical heterogeneity.

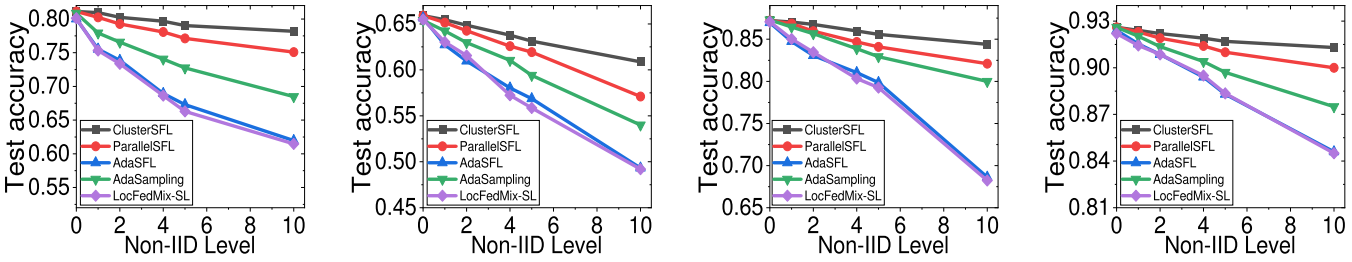
Thirdly, to illustrate the advantage of ClusterSFL in saving communication resource, we illustrate the network traffic consumption of these approaches when achieving different target accuracies in Fig. 5. By the results, the network traffic consumption of all approaches increases with the target accuracy for all the four datasets. Furthermore, ClusterSFL with feature compression always consumes the least network traffic among all approaches. In addition, model splitting (*i.e.*, ClusterSFL, ParallelSFL, AdaSFL and LocFedMix-SL) helps to save much more network traffic compared to typical FL approaches such as AdaSampling. Both AdaSFL with adaptive local updating frequency and ParallelSFL with adaptive worker clustering reduce the network traffic consumption, while ClusterSFL with feature compression further reduces the network traffic

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:02:24 UTC from IEEE Xplore. Restrictions apply.




(a) CIFAR-10 (b) IMAGE-100 (c) Speech (d) QNLI

Fig. 6. Average waiting time of five approaches on the four datasets.




(a) CIFAR-10 (b) IMAGE-100 (c) Speech (d) QNLI

Fig. 7. Test accuracy varies with different non-IID levels.

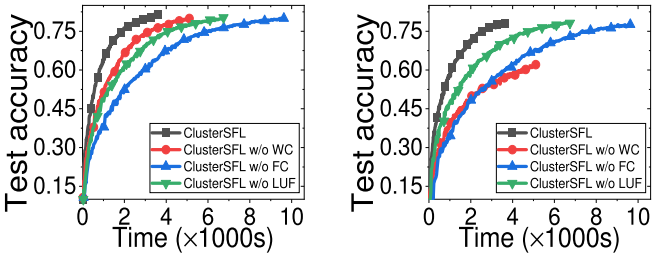
consumption. Specifically, by Fig. 5(a), for training AlexNet on CIFAR-10 to achieve 81% accuracy, ClusterSFL reduces the network traffic consumption by about 2,071MB, 5,083MB, 22,089MB and 11,187MB, compared to ParallelSFL, AdaSFL, AdaSampling and LocFedMix-SL, respectively. Besides, as illustrated in Fig. 5(b), ClusterSFL saves network traffic consumption by about 22%, 38%, 63% and 55%, when achieving 65% accuracy for VGG16 on IMAGE-100, compared to the baselines (*i.e.*, ParallelSFL, AdaSFL, AdaSampling, and LocFedMix-SL). Notably, as shown in Fig. 5(c), when achieving 87% accuracy, ParallelSFL, ClusterSFL, AdaSFL and LocFedMix-SL consume 894MB, 1,192MB, 1,692MB and 2,398MB, respectively, while AdaSampling consumes 3,403MB for CNN on Speech. Moreover, by Fig. 5(d), for training RoBERTa on QNLI to achieve 92% accuracy, ClusterSFL reduces the network traffic consumption by about 3,320MB, 7,103MB, 28,067MB and 17,857MB, compared to ClusterSFL, AdaSFL, AdaSampling and LocFedMix-SL, respectively. These results demonstrate that ClusterSFL is effective in saving network traffic consumption.

Finally, to further demonstrate the robustness of ClusterSFL towards system heterogeneity, we illustrate the average waiting time of the five approaches on both datasets in Fig. 6. Both ClusterSFL with adaptive diverse feature compression ratios for heterogeneous workers and AdaSFL with adaptive and diverse batch sizes for heterogeneous workers achieves the least waiting time, but the waiting time of ParallelSFL is close to AdaSFL and is much less than that of other approaches. For instance, by Fig. 6(a), compared to ParallelSFL, AdaSampling and LocFedMix-SL, ClusterSFL reduces the average waiting time to train AlexNet on CIFAR-10 by about 25%, 70% and 62%, respectively. Considering the workers with varying capacities, LocFedMix-SL does not consider system heterogeneity, thus leading to non-negligible waiting time. AdaSampling with adaptive worker sampling reduces the average waiting time to a certain extent. Specifically, by Fig. 6(b), ClusterSFL reduces the average waiting time for VGG16 on IMAGE-100 by about 30%, 65%, and 81%, compared to ParallelSFL, AdaSampling and LocFedMix-SL, respectively. Similarly, as shown in Fig. 6(c), the average waiting time of ClusterSFL is 1.3s for CNN on Speech while ParallelSFL, AdaSFL, AdaSampling and LocFedMix-SL incur that of 1.5s, 1.3s, 4.4s and 7.3s, respectively. Moreover, by Fig. 6(d), the average waiting time of ClusterSFL is 9.7s for RoBERTa on QNLI while ParallelSFL, AdaSFL, AdaSampling and LocFedMix-SL incur an average waiting time of 10.1s, 9.6s, 34.5s and 53.7s, respectively. In general, these results illustrate that ClusterSFL with effective cluster clustering and adaptive feature compression overcomes the challenge of system heterogeneity well, compared to existing methods.

### C. Effect of Non-IID Levels

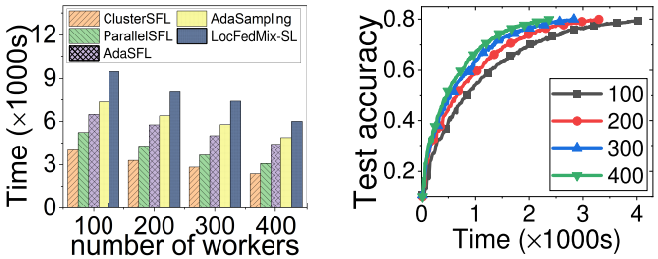
To demonstrate the effectiveness of ClusterSFL in handling non-IID data, we present the test accuracy of different approaches at varying non-IID levels in Fig. 7, in which the model accuracy of the five approaches on all the datasets decreases as the non-IID level increases. However, ClusterSFL and ParallelSFL consistently outperform the other approaches on all datasets. AdaSFL and LocFedMix-SL, without considering the challenges of statistical heterogeneity, exhibit the lowest model accuracy on non-IID datasets. To tackle statistical heterogeneity, AdaSampling mitigates the impact of non-IID data on model training to some extent by designing an adaptive worker sampling algorithm, and ParallelSFL with effective clustering strategy overcomes the statistical

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:02:24 UTC from IEEE Xplore. Restrictions apply.




(a) IID (b) Non-IID

Fig. 8. Effects of worker clustering and local updating frequency optimization.




(a) Completion Time (b) Training Process

Fig. 9. Performance comparison with different number of workers.

heterogeneity better than AdaSampling. By introducing the strategy of feature mixing, ClusterSFL always achieves the highest accuracy in the in the non-IID scenario. Specifically, by Fig. 7(a), with non-IID level of $p$=10 on CIFAR-10, ClusterSFL achieves an improvement of final accuracy by about 4.11%, 26.16%, 14.16%, 27.18%, compared to the baselines (*i.e.*, ParallelSFL, AdaSFL, AdaSampling, LocFedMix-SL). Notably, as shown in Fig. 7(b), while transitioning from IID to non-IID level of $p$=10 on IMAGE-100, ClusterSFL, ParallelSFL and AdaSampling suffer from only 7.62%, 13.35% and 17.41% loss in accuracy, while the accuracy loss for AdaSFL, and LocFedMix-SL is 24.89% and 24.86%, respectively. Besides, as illustrated in Fig. 7(c), ClusterSFL can achieve improvement of test accuracy by about 2.78%, 22.94%, 5.39% and 23.68% on Speech with non-IID level of $p = 10$, compared to the baselines (*i.e.*, ParallelSFL, AdaSFL, AdaSampling and LocFedMix-SL). Moreover, by Fig. 7(d), with non-IID level of $p = 10$ on QNLI, ClusterSFL, ParallelSFL and AdaSampling achieve 91.68%, 90.11% and 87.33% accuracy, while LocFedMix-SL and AdaSFL only achieve 84.51% and 84.62% accuracy. Collectively, these results further demonstrate the advantage of ClusterSFL with effective cluster clustering and feature mixing in addressing statistical heterogeneity.

### D. Effect of Key Strategies

There are three key strategies of ClusterSFL, *i.e.*, worker clustering with feature mixing, feature compression ratio configuration, and local updating frequency optimization, being developed to enhance the performance of SFL. Herein, we conduct several sets of experiments for training AlexNet on CIFAR-10 with IID distribution ($p$=0) and non-IID distribution ($p$=10) to evaluate the effectiveness of the three critical strategies in ClusterSFL. We adopt the ClusterSFL without worker clustering with feature mixing (denoted as ClusterSFL w/o WC), ClusterSFL without feature compression ratio configuration (denoted as ClusterSFL w/o FC) and ClusterSFL without local updating frequency optimization (denoted as ClusterSFL w/o LUF) as the baselines. Concretely, in ClusterSFL w/o WC, the PS randomly partitions the workers into several clusters and determines the top worker for each cluster. In ClusterSFL w/o FC, all the bottom workers don't compress their features, while in ClusterSFL w/o LUF, the PS assigns the identical and fixed local updating frequency for each cluster. By Fig. 8, ClusterSFL w/o WC converges much faster than ClusterSFL w/o FC and ClusterSFL w/o LUF on the IID dataset, while ClusterSFL w/o FC and ClusterSFL w/o LUF achieves higher test accuracy than ClusterSFL w/o WC on the non-IID dataset. Specifically, ClusterSFL reduces the total training time by about 29%, 62%, 46%, compared to ClusterSFL w/o WC, ClusterSFL w/o FC, ClusterSFL w/o LUF, respectively, illustrating the advantage of feature compression ratio configuration and local updating frequency optimization in dealing with the system heterogeneity. Besides, ClusterSFL improves the final test accuracy by about 25.78% on the non-IID dataset, compared to ClusterSFL w/o WC, which demonstrates the importance of worker clustering with feature mixing in addressing statistical heterogeneity. These experimental results reflect the positive roles of three key strategies in ClusterSFL.

### E. Effect of System Scales

In this section, to demonstrate the robustness of ClusterSFL, we evaluate the performance of ClusterSFL and baselines with different scales of participating workers. We conduct several sets of experiments for training AlexNet on CIFAR-10 with four scales (*e.g.*, 100, 200, 300, 400) through extensive simulation experiments, which are conducted on an AMAX deep learning workstation equipped with an Intel(R) Xeon(R) Platinum 8358P CPU @ 2.60GHz, 4 NVIDIA GeForce RTX A6000 GPUs (48GB GPU memory each) and 512 GB RAM. The results of completion time to achieve 80% accuracy for these approaches are presented in Fig. 9(a), while the training processes of different scales for ClusterSFL are presented in Fig. 9(b). As the number of participating workers increases, all approaches achieve faster convergence. The reason is that the number of samples on a worker is limited and more workers contribute more local data for model training in each round, thus speeding up model training. For instance, ClusterSFL with 400 workers reduces the total training time by about 39%, 26%, 17%, compared to ClusterSFL with 100, 200, and 300 workers, respectively. In addition, ClusterSFL also achieves a speedup of $1.31\times \sim 2.19\times$ to reach the target accuracy, compared to the baselines (*i.e.*, ParallelSFL, AdaSFL, AdaSampling and LocFedMix-SL) regarding the different scales of participating workers. These results further illustrate the robustness and advantage of ClusterSFL.

## VI. Conclusion

In this paper, we have proposed a novel SFL framework, named ClusterSFL, which integrates the advantages of FL and SFL and is designed to tackle heterogeneity issues by

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:02:24 UTC from IEEE Xplore. Restrictions apply.

effective worker clustering and adaptive feature compression. Under the heterogeneity restrictions, ClusterSFL defines the utility functions with the heterogeneity restrictions to serve as the comprehensive metrics for estimating waiting time and data distribution of worker clusters, and develops the clustering strategy and feature compression ratio configuration to balance the trade-off between training efficiency and model accuracy. Meanwhile, ClusterSFL assigns diverse and appropriate local updating frequencies for each cluster to further address system heterogeneity. The experimental results show that ClusterSFL can reduce the traffic consumption by about 22%$\sim$63%, speed up the model training by about $1.29\times \sim 2.83\times$, and improve the final model accuracy by about 2%$\sim$27% in heterogeneous scenarios, compared to the baselines.

Although ClusterSFL has demonstrated significant success in addressing heterogeneity issues and achieved high performance of model training, concerns regarding the privacy of data and model still persist. We provide a detailed discussion of the potential privacy risks associated with ClusterSFL in Sec. IV-F, and in future work, we would pay more attention to enhancing privacy protection mechanisms for both data and models in SFL. Moreover, ClusterSFL is primarily designed for typical CNN-based architectures, while large language models (LLMs) based on transformer architectures are gaining increasing attention. As part of our future work, we aim to explore how to implement the training and fine-tuning of LLM and improve the performance within the SFL framework.

## REFERENCES


[1] Y. Liao, Y. Xu, H. Xu, Z. Yao, L. Huang, and C. Qiao, "ParallelSFL: A novel split federated learning framework tackling heterogeneity issues," in *Proc. 30th Annu. Int. Conf. Mobile Comput. Netw.*, Dec. 2024, pp. 845–860.

[2] B. L. R. Stojkoska and K. V. Trivodaliev, "A review of Internet of Things for smart home: Challenges and solutions," *J. Cleaner Prod.*, vol. 140, pp. 1454–1464, Jan. 2017.

[3] S. Bansal and D. Kumar, "IoT ecosystem: A survey on devices, gateways, operating systems, middleware and communication," *Int. J. Wireless Inf. Netw.*, vol. 27, no. 3, pp. 340–364, Sep. 2020.

[4] D. Yacchirema, J. S. de Puga, C. Palau, and M. Esteve, "Fall detection system for elderly people using IoT and big data," *Proc. Comput. Sci.*, vol. 130, pp. 603–610, Jan. 2018.

[5] N. Komalapati, V. C. Yarra, L. A. V. Kancharla, and T. N. Shankar, "Smart fire detection and surveillance system using IoT," in *Proc. Int. Conf. Artif. Intell. Smart Syst. (ICAIS)*, Mar. 2021, pp. 1386–1390.

[6] M. F. Elrawy, A. I. Awad, and H. F. A. Hamed, "Intrusion detection systems for IoT-based smart environments: A survey," *J. Cloud Comput.*, vol. 7, no. 1, pp. 1–20, Dec. 2018.

[7] R. Rudman and R. Bruwer, "Defining web 3.0: Opportunities and challenges," *Electron. Library*, vol. 34, no. 1, pp. 132–154, Feb. 2016.

[8] E. Li, L. Zeng, Z. Zhou, and X. Chen, "Edge AI: On-demand accelerating deep neural network inference via edge computing," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 447–457, Jan. 2020.

[9] A. Hard et al., "Federated learning for mobile keyboard prediction," 2018, *arXiv:1811.03604*.

[10] P. Kairouz et al., "Advances and open problems in federated learning," 2019, *arXiv:1912.04977*.

[11] R. Han, Q. Zhang, C. H. Liu, G. Wang, J. Tang, and L. Y. Chen, "LegoDNN: Block-grained scaling of deep neural networks for mobile vision," in *Proc. Annu. Int. Conf. Mobile Comput. Netw. (MobiCom)*, 2021, pp. 406–419.

[12] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, vol. 54, A. Singh and J.Zhu., Eds., Fort Lauderdale, FL, USA, 2017, pp. 1273–1282.

[13] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.

[14] P. Nakkiran, G. Kaplun, Y. Bansal, T. Yang, B. Barak, and I. Sutskever, "Deep double descent: Where bigger models and more data hurt," *J. Stat. Mech., Theory Exp.*, vol. 2021, no. 12, Dec. 2021, Art. no. 124003.

[15] C. Niu et al., "Billion-scale federated learning on mobile clients: A submodel design with tunable privacy," in *Proc. 26th Annu. Int. Conf. Mobile Comput. Netw.*, 2020, pp. 1–14.

[16] W. Zhang et al., "ELF: Accelerate high-resolution mobile deep vision with content-aware parallel offloading," in *Proc. 27th Annu. Int. Conf. Mobile Comput. Netw.*, 2021, pp. 201–214.

[17] J. Park et al., "Communication-efficient and distributed learning over wireless networks: Principles and applications," *Proc. IEEE*, vol. 109, no. 5, pp. 796–819, May 2021.

[18] S. Pal et al., "Server-side local gradient averaging and learning rate acceleration for scalable split learning," 2021, *arXiv:2112.05929*.

[19] Y. Liao, Y. Xu, H. Xu, L. Wang, Z. Yao, and C. Qiao, "MergeSFL: Split federated learning with feature merging and batch size regulation," in *Proc. IEEE 40th Int. Conf. Data Eng. (ICDE)*, May 2024, pp. 2054–2067.

[20] S. Oh et al., "LocFedMix-SL: Localize, federate, and mix for improved scalability, convergence, and latency in split learning," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 3347–3357.

[21] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, "SplitFed: When federated learning meets split learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, 2022, pp. 8485–8493.

[22] D.-J. Han, H. I. Bhatti, J. Lee, and J. Moon, "Accelerating federated learning with split learning on locally generated losses," in *Proc. Workshop Federated Learn. User Privacy Data Confidentiality ICML Board*, 2021, pp. 1–21.

[23] A. Abedi and S. S. Khan, "FedSL: Federated split learning on distributed sequential data in recurrent neural networks," 2020, *arXiv:2011.03180*.

[24] F. Sattler, K.-R. Müller, and W. Samek, "Clustered federated learning: Model-agnostic distributed multitask optimization under privacy constraints," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 8, pp. 3710–3722, Aug. 2021.

[25] C. Li, X. Zeng, M. Zhang, and Z. Cao, "PyramidFL: A fine-grained client selection framework for efficient federated learning," in *Proc. 28th Annu. Int. Conf. Mobile Comput. Netw.*, Oct. 2022, pp. 158–171.

[26] B. Luo, W. Xiao, S. Wang, J. Huang, and L. Tassiulas, "Tackling system and statistical heterogeneity for federated learning with adaptive client sampling," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2022, pp. 1739–1748.

[27] Y. Liao, Y. Xu, H. Xu, Z. Yao, L. Wang, and C. Qiao, "Accelerating federated learning with data and model parallelism in edge computing," *IEEE/ACM Trans. Netw.*, vol. 32, no. 1, pp. 904–918, Feb. 2024.

[28] W. Zhuang, Y. Wen, and S. Zhang, "Divergence-aware federated self-supervised learning," 2022, *arXiv:2204.04385*.

[29] L. Wang, Y. Xu, H. Xu, M. Chen, and L. Huang, "Accelerating decentralized federated learning in heterogeneous edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 9, pp. 5001–5016, 2022.

[30] Y. Xie et al., "FederatedScope: A flexible federated learning platform for heterogeneity," *Proc. VLDB Endowment*, vol. 16, no. 5, pp. 1059–1072, Jan. 2023.

[31] Y. Liao, Y. Xu, H. Xu, L. Wang, C. Qian, and C. Qiao, "Decentralized federated learning with adaptive configuration for heterogeneous participants," *IEEE Trans. Mobile Comput.*, vol. 23, no. 6, pp. 7453–7469, Jun. 2024.

[32] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," 2018, *arXiv:1806.00582*.

[33] H. Wang, Z. Kaplan, D. Niu, and B. Li, "Optimizing federated learning on non-IID data with reinforcement learning," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, Jul. 2020, pp. 1698–1707.

[34] S. Chen, Y. Xu, H. Xu, Z. Jiang, and C. Qiao, "Decentralized federated learning with intermediate results in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 341–358, Jan. 2024.

[35] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "Oort: Efficient federated learning via guided participant selection," in *Proc. OSDI*, 2020, pp. 19–35.

[36] Y. Xu, Y. Liao, H. Xu, Z. Ma, L. Wang, and J. Liu, "Adaptive control of local updating and model compression for efficient federated learning," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 5675–5689, Oct. 2023.

[37] W. Y. B. Lim et al., "Federated learning in mobile edge networks: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 3, pp. 2031–2063, 3rd Quart., 2020.

[38] E. Diao, J. Ding, and V. Tarokh, "HeteroFL: Computation and communication efficient federated learning for heterogeneous clients," in


Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:02:24 UTC from IEEE Xplore. Restrictions apply.

*Proc. Int. Conf. Learn. Represent.*, 2021, pp. 1–24. [Online]. Available: https://openreview.net/forum?id=TNkPBBYFkXg

[39] B. Liu, W. Shen, P. Li, and X. Zhu, "Accelerate mini-batch machine learning training with dynamic batch size fitting," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2019, pp. 1–8.

[40] Z. Ma, Y. Xu, H. Xu, Z. Meng, L. Huang, and Y. Xue, "Adaptive batch size for federated learning in resource-constrained edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 1, pp. 37–53, Jan. 2023.

[41] J. Shin, Y. Li, Y. Liu, and S.-J. Lee, "Fedbalancer: Data and pace control for efficient federated learning on heterogeneous clients," in *Proc. Int. Conf. Mobile Syst. Appl. Services (MobiSys)*, 2022, pp. 436–449.

[42] S. Vahidian et al., "Efficient distribution similarity identification in clustered federated learning via principal angles between client data subspaces," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 10043–10052.

[43] A. Ghosh, J. Chung, D. Yin, and K. Ramchandran, "An efficient framework for clustered federated learning," *IEEE Trans. Inf. Theory*, vol. 68, no. 12, pp. 8076–8091, Dec. 2022.

[44] O. Gupta and R. Raskar, "Distributed learning of deep neural network over multiple agents," *J. Netw. Comput. Appl.*, vol. 116, pp. 1–8, Aug. 2018.

[45] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, "Split learning for health: Distributed deep learning without sharing raw patient data," 2018, *arXiv:1812.00564*.

[46] M. Kim, A. DeRieux, and W. Saad, "A bargaining game for personalized, energy efficient split learning over wireless networks," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Glasgow, U.K., May 2023, pp. 1–6.

[47] P. Joshi, C. Thapa, S. Camtepe, M. Hasanuzzamana, T. Scully, and H. Afli, "Splitfed learning without client-side synchronization: Analyzing client-side split network portion size to overall performance," 2021, *arXiv:2109.09246*.

[48] T. Castiglia, A. Das, S. Wang, and S. Patterson, "Compressed-VFL: Communication-efficient learning with vertically partitioned data," in *Proc. 39th Int. Conf. Mach. Learn.*, 2022, pp. 2738–2766.

[49] A. Ayad, M. Renner, and A. Schmeink, "Improving the communication and computation efficiency of split learning for IoT applications," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2021, pp. 01–06.

[50] F. Zheng, C. Chen, L. Lyu, and B. Yao, "Reducing communication for split learning by randomized top-k sparsification," in *Proc. 32nd Int. Joint Conf. Artif. Intell.*, Aug. 2023, pp. 4665–4673.

[51] D.-J. Han et al., "Federated split learning with joint personalization-generalization for inference-stage optimization in wireless edge networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 6, pp. 7048–7065, Jun. 2024.

[52] S. Wang et al., "When edge meets learning: Adaptive control for resource-constrained distributed machine learning," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2018, pp. 63–71.

[53] D. Leroy, A. Coucke, T. Lavril, T. Gisselbrecht, and J. Dureau, "Federated learning for keyword spotting," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2019, pp. 6341–6345.

[54] X. Lyu, C. Ren, W. Ni, H. Tian, R. P. Liu, and Y. J. Guo, "Multi-timescale decentralized online orchestration of software-defined networks," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 12, pp. 2716–2730, Dec. 2018.

[55] X. Lyu, C. Ren, W. Ni, H. Tian, R. P. Liu, and E. Dutkiewicz, "Optimal online data partitioning for geo-distributed machine learning in edge of wireless networks," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 10, pp. 2393–2406, Oct. 2019.

[56] J. R. Hershey and P. A. Olsen, "Approximating the Kullback Leibler divergence between Gaussian mixture models," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2007, p. 317.

[57] Goldberger, Gordon and Greenspan, "An efficient image similarity measure based on approximations of KL-divergence between two Gaussian mixtures," in *Proc. 9th IEEE Int. Conf. Comput. Vis.*, vol. 1, Oct. 2003, pp. 487–493, doi: 10.1109/ICCV.2003.1238387.

[58] S. Li, E. Ngai, F. Ye, and T. Voigt, "Auto-weighted robust federated learning with corrupted data sources," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 5, pp. 1–20, Oct. 2022.

[59] Z. Li, T. Lin, X. Shang, and C. Wu, "Revisiting weighted aggregation in federated learning with neural networks," 2023, *arXiv:2302.10911*.

[60] C. Thapa, M. A. P. Chamikara, and S. A. Camtepe, "Advancements of federated learning towards privacy preservation: From federated learning to split learning," in *Federated Learning Systems: Towards Next-Generation AI*. Cham, Switzerland: Springer, 2021, pp. 79–109.

[61] M. Wu et al., "Federated split learning with data and label privacy preservation in vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 1, pp. 1223–1238, Jan. 2024.

[62] B. Ghazi, N. Golowich, R. Kumar, P. Manurangsi, and C. Zhang, "Deep learning with label differential privacy," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 27131–27145.

[63] Z. Yang et al., "Dynamic corrected split federated learning with homomorphic encryption for U-shaped medical image networks," *IEEE J. Biomed. Health Informat.*, vol. 27, no. 12, pp. 5946–5957, Dec. 2023.

[64] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux J.*, vol. 2014, no. 239, p. 2, 2014.

[65] N. Naik, "Building a virtual system of systems using Docker swarm in multiple clouds," in *Proc. IEEE Int. Symp. Syst. Eng. (ISSE)*, Oct. 2016, pp. 1–3.

[66] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. 33rd Conf. Neural Inf. Process. Syst.*, 2019, pp. 8026–8037.

[67] E. Gabriel et al., "Open MPI: Goals, concept, and design of a next generation MPI implementation," in *European Parallel Virtual Machine/Message Passing Interface Users Group Meeting*. Cham, Switzerland: Springer, 2004, pp. 97–104.

[68] A. Tirumala. (1999). *IPerf—The Ultimate Speed Test Tool for TCP, UDP and SCTP*. [Online]. Available: http://dast.nlanr.net/Projects/Iperf/

[69] A. Krizhevsky and G. Hinton, "Convolutional deep belief networks on CIFAR-10," *Unpublished Manuscript*, vol. 40, no. 7, pp. 1–9, 2010.

[70] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 2012, pp. 1097–1105.

[71] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, Dec. 2015.

[72] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.

[73] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," 2018, *arXiv:1804.03209*.

[74] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," 2018, *arXiv:1804.07461*.

[75] T.-M. Harry Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," 2019, *arXiv:1909.06335*.

[76] M. Yurochkin, M. Agarwal, S. Ghosh, K. Greenewald, N. Hoang, and Y. Khazaeni, "Bayesian nonparametric federated learning of neural networks," in *Proc. Int. Conf. Mach. Learn.*, vol. 97, 2019, pp. 7252–7261.

[77] Y. Liao, Y. Xu, H. Xu, L. Wang, and C. Qian, "Adaptive configuration for heterogeneous participants in decentralized federated learning," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2023, pp. 1–10.

[78] D. Rothchild et al., "FetchSGD: Communication-efficient federated learning with sketching," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 8253–8265.

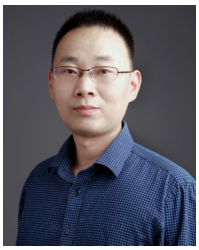


**Yang Xu** (Member, IEEE) received the B.S. degree from Wuhan University of Technology in 2014 and the Ph.D. degree in computer science and technology from the University of Science and Technology of China in 2019. He is currently an Associate Researcher with the School of Computer Science and Technology, University of Science and Technology of China. His primary research interests include the Internet of Things, edge computing, federated learning, and edge intelligence.



**Yunming Liao** received the B.S. degree from the University of Science and Technology of China in 2020, where he is currently pursuing the Ph.D. degree with the School of Computer Science and Technology. His research interests include mobile edge computing, federated learning, and edge intelligence.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:02:24 UTC from IEEE Xplore. Restrictions apply.



**Hongli Xu** (Member, IEEE) received the B.S. degree in computer science and the Ph.D. degree in computer software and theory from the University of Science and Technology of China, China, in 2002 and 2007, respectively. He is currently a Professor with the School of Computer Science and Technology, University of Science and Technology of China. He has published more than 100 papers in famous journals and conferences, including IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, Infocom, and ICNP. He has also held more than 30 patents. His main research interests include software defined networks, edge computing, and the Internet of Thing. He has won the best paper award or the best paper candidate in several famous conferences. He was awarded the Outstanding Youth Science Foundation of NSFC in 2018.



**Zhiwei Yao** received the B.S. degree from Hohai University in 2021. He is currently pursuing the Ph.D. degree with the School of Computer Science and Technology, University of Science and Technology of China. His main research interests include edge computing and federated learning.



**Junhao Cheng** received the B.S. degree from Hangzhou Dianzi University in 2023. He is currently pursuing the M.S. degree with the School of Computer Science and Technology, University of Science and Technology of China. His main research interests include edge computing and federated learning.



**Chunming Qiao** (Fellow, IEEE) is currently a SUNY Distinguished Professor and also the Current Chair of the Computer Science and Engineering Department, University at Buffalo, The State University of New York, Buffalo, NY, USA. His current focus is on connected and autonomous vehicles. He has published extensively with an H-index of more than 69. He also has seven U.S. patents and has worked as a Consultant for several IT and telecommunications companies since 2000. His research has been funded by a dozen major IT and telecommunications companies, including Cisco and Google, and more than a dozen NSF grants. He was elected to an IEEE Fellow "for his contributions to optical and wireless network architectures and protocols." Two of his papers have received the Best Paper Awards from IEEE and Joint ACM/IEEE venues.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:02:24 UTC from IEEE Xplore. Restrictions apply.